Week 6 Lab – From Generated Tests to a CI Gate

This week we will let an agent write a test suite and then decide whether to accept it. Generating tests is close to free, but determining whether they would catch an actual fault is not.

After creating the tests, we push the code to GitHub, turn that suite into a CI workflow, and make it a gate, i.e. a pull request that breaks a test cannot be merged.

Week 5's lab got your code into a repository behind a protected main branch. Today we add the piece that makes the protection mean something: a required status check.

Before You Start

1. An **agentic coding tool** of your choice (GitHub Copilot, Cursor, Claude Code, Gemini CLI, …). If yours misbehaves today, generate the tests in a plain chat assistant (e.g. SMU's Gemini subscription) and paste them in.
2. Python 3 and Git installed locally (`git --version` to check), and a GitHub account.
3. Nothing to install beyond that, and **nothing to submit**.

The Class Under Test

> **QuackLoan, continued.** The rubber-duck lending library has started charging late fees. The first two days late are forgiven. After that it is 50 cents for each further day, doubled for a Deluxe duck, and a single fine never exceeds $5. Fines accumulate against the member's account.

Copy the class below into a file called `duckfine.py`. That is the whole system under test: four rules, one method.

```
class DuckFine:
    """Late fees for the QuackLoan rubber-duck lending library."""

    DAILY_FEE  = 0.50   # dollars per chargeable day
    GRACE_DAYS = 2      # the first two days late are forgiven
    MAX_FEE    = 5.00   # a single fine never exceeds this

    def __init__(self, member_id):
        self.member_id  = member_id
        self.total_owed = 0.0

    def charge(self, days_late, deluxe=False):
        if days_late < 0:
            raise ValueError("days_late must not be negative")
        chargeable = max(0, days_late - self.GRACE_DAYS)
        fee = chargeable * self.DAILY_FEE
        if deluxe:
            fee *= 2
        fee = min(fee, self.MAX_FEE)
        self.total_owed += fee
        return fee
```

Part 1 – Let the Agent Write the Tests

4. Create an empty folder, save `duckfine.py` into it, and open the folder in your agentic tool.
5. Ask for a test suite. Something like:

   Write unit tests for the DuckFine class in duckfine.py. Use Python's unittest, one behaviour per test method, in a file called test_duckfine.py.

6. Run them:

   ```
   python -m unittest -v
   ```

   They pass. That tells you the tests agree with the code; it does not tell you the code is right. (Short of time, or the AI tool is not cooperating? A typical generated suite is printed in **Appendix A** of this lab. Use that one and go straight to Part 2.)

Part 2 – Decide Whether to Believe Them

7. Read the suite against the five review questions from today's class. In particular: for each test, what wrong implementation would make it fail? If the answer is "nothing plausible", the test is nothing more than decoration.
8. Measure coverage:

   ```
   python -m pip install coverage
   coverage run --branch -m unittest
   coverage report -m
   ```

   Write the number down, then set it aside for a moment.

9. Now seed some faults (mutants). For each row below, make that one change to `duckfine.py`, re-run the suite, note whether anything failed, and undo the change before the next row.

#	Change to duckfine.py	Did any test fail?
1	Remove the guard: `chargeable = days_late - self.GRACE_DAYS`	yes
2	Shorten the grace period: `GRACE_DAYS = 1`	yes
3	Drop the cap: delete the line `fee = min(fee, self.MAX_FEE)`	yes
4	Forget the running total: delete `self.total_owed += fee`	yes
5	Overcharge for Deluxe: `fee *= 3`	yes

10. Expect a mix – two or three caught, the rest waved through. **A fault that nothing fails on is a requirement that nothing tests.** Write those missing tests yourself, one per uncaught fault, named for the requirement rather than the method (e.g. `test_fine_is_capped_at_five_dollars`, not `test_charge_3`). Take the expected values from the brief, not from running the code: a value copied out of the implementation can only ever confirm what the implementation already does. (If your suite caught all five, it is a good one already!)
11. Re-run the five rows. All five should now fail something. Then re-run coverage: the number has barely moved, if at all – it was most likely 100% before you started. Coverage told you which lines ran but it never told you what was checked.

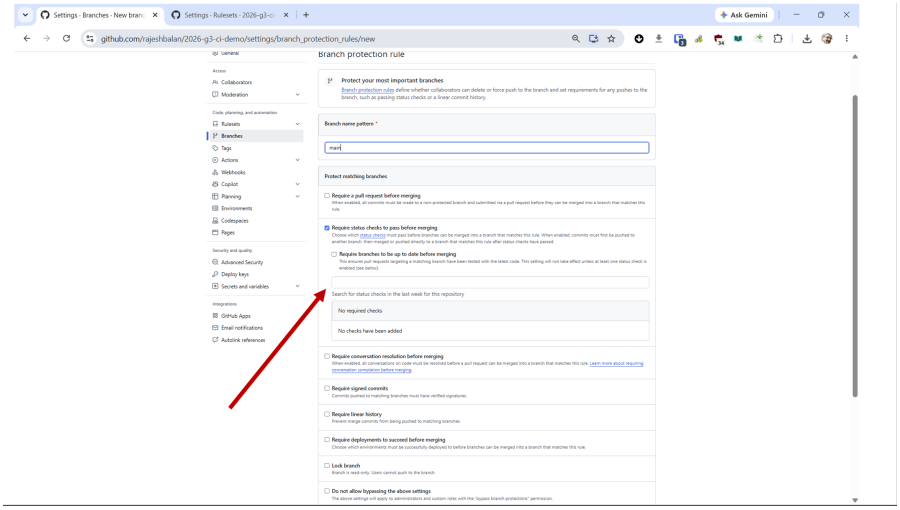
Part 3 – A Repository, a Protected main, and a Pipeline

12. Create a new repository called `week6-lab` on GitHub (https://github.com/new) and make it **public**. Branch protection rules and rulesets are free on public repositories; on a private repository they need a paid plan, and public repositories also get unlimited Actions minutes. (Claim the **GitHub Student Developer Pack** and you get GitHub Pro, which protects private repositories too – worth doing before your project needs it.)
13. Push `duckfine.py` and `test_duckfine.py` to it – command line, GitHub Desktop, or VS Code, whichever you used in Week 5. Add a Python `.gitignore` so caches stay out of the repository.
14. Ask your agent for the pipeline:

    Add a GitHub Actions workflow at .github/workflows/tests.yml that runs on pushes to main and on pull requests, sets up Python 3.13, and runs python -m unittest discover. Name the job "test".

    Read the YAML before you trust it: it is code, it lives in the repository, and it is reviewed like code. A reference workflow is on the last page **(Appendix B)**.

15. Commit and push the workflow. Open the **Actions** tab and watch the run: clean runner, fresh checkout, your tests. Green tick.
16. Now make it a gate. Go to **Settings → Branches → Add classic branch protection** (newer accounts: **Settings → Rulesets → New branch ruleset**), targeting `main`, and turn on:
    a. **Require a pull request before merging**
    b. **Require status checks to pass** — then click the search bar below it, search for test and select `test`



Leave **Require approvals** off, as in Week 5: you still need to be able to merge your own pull request. Note the ordering: `test` only appears in that list because the workflow has already run once. Only ticking the box shifts CI from advisory to a gate.

Part 4 – Watch the Gate Hold

17. Create a short-lived branch and reintroduce one of the faults your new tests now catch. Dropping the cap (row 6) is a good choice: it looks like a simplification rather than a mistake, which is exactly how these arrive in real pull requests.

    ```
    git switch -c fine-tuning
    ```

18. Commit the change, push the branch, and open a pull request into `main`.
19. Watch the check run. It fails, and the merge button is disabled (not by convention, but by configuration!). Click **Details**: the failing assertion names the requirement you broke. Without this ruleset, the check would have gone red and you could have merged anyway.
20. Fix the code on the branch and push. Watch the same check go green, merge the pull request, and delete the branch.

The test that stopped that merge was not one the agent wrote. It was the one you added after asking what would have to break for a test to fail. That is the whole of this lab: the agent produced the suite, you decided what counted as evidence, and the pipeline now enforces that decision on everyone (a teammate, a future you at 2am, or an agent opening a pull request on your behalf).

Nothing to submit. Set the same gate up on your project repository this week; the workflow file will barely change. If a run goes red in a way you cannot explain, reach out to us on Slack!

Appendix A – A Typical Generated Suite

If you skipped Part 1, start from this. It is close to what agents produce for `DuckFine` unprompted: it passes, it is readable, and it reports 100% statement and branch coverage. Three of the five faults are missed by it.

```
import unittest
from duckfine import DuckFine


class TestDuckFine(unittest.TestCase):

    def setUp(self):
        self.fine = DuckFine("M001")

    def test_charge_returns_a_fee_for_a_late_duck(self):
        self.assertEqual(self.fine.charge(5), 1.50)

    def test_deluxe_duck_costs_more(self):
        self.assertTrue(self.fine.charge(5, deluxe=True) > 0)

    def test_no_fee_when_returned_on_time(self):
        self.assertEqual(self.fine.charge(0), 0.0)

    def test_negative_days_raises(self):
        with self.assertRaises(ValueError):
            self.fine.charge(-1)


if __name__ == "__main__":
    unittest.main()
```

Appendix B – A Reference Workflow

Save as `.github/workflows/tests.yml` if you would rather not generate it. The job id — `test` — is the name that appears in the status-check list in Part 3.

```
name: tests

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - name: Run unit tests
        run: python -m unittest discover -v
```